

Observer pattern

1. Название и классификация

Название: Наблюдатель (Observer)

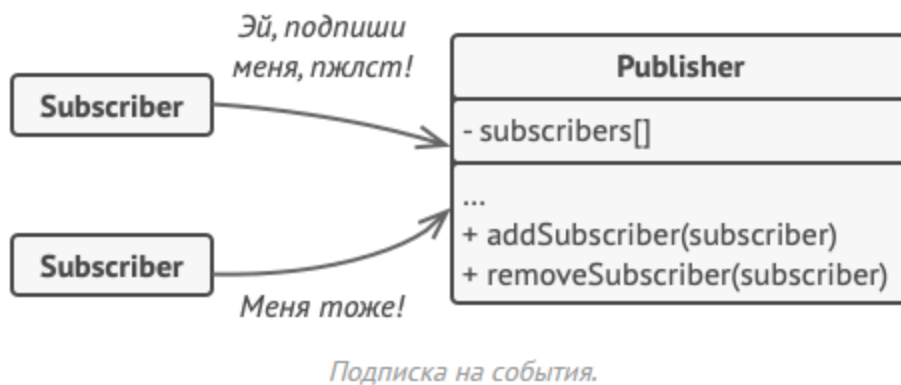
Также известен как: Издатель-Подписчик, Слушатель

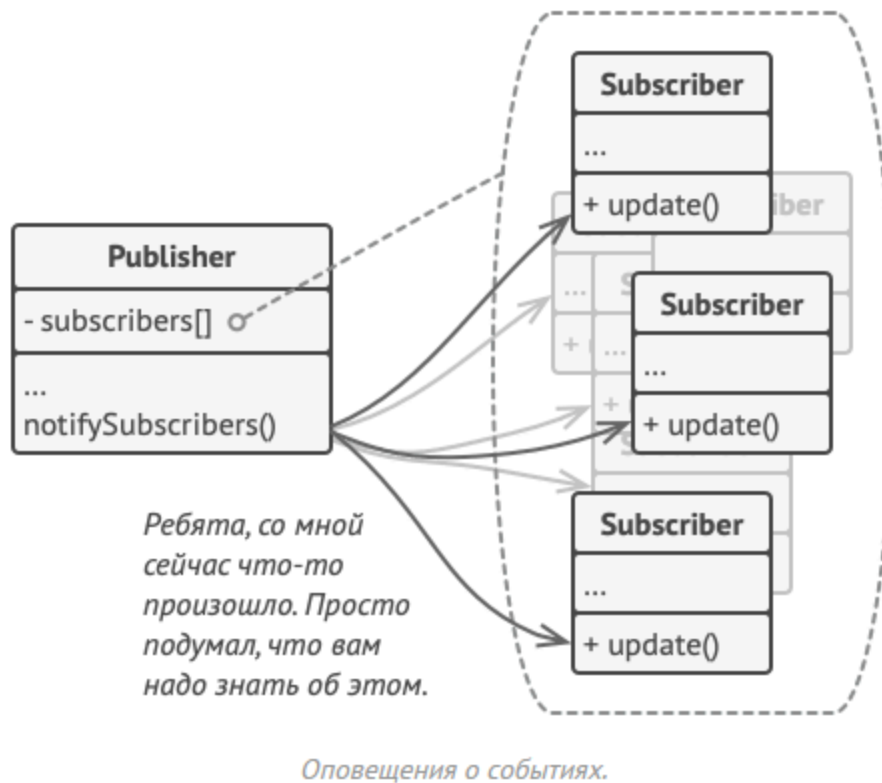
Классификация: Поведенческий паттерн

2. Назначение

Создаёт механизм подписки, чтобы одни объекты следили за изменениями в других. Издатель оповещает подписчиков, не зная, кто именно подписан.

3. Мотивация





Проблема: Покупатель ходит в магазин каждый день (теряет время) или магазин рассылает спам всем (теряет ресурсы).

Решение: Магазин (издатель) ведёт список подписчиков. Покупатель подписывается и получает уведомления только когда товар появился.

4. Применимость

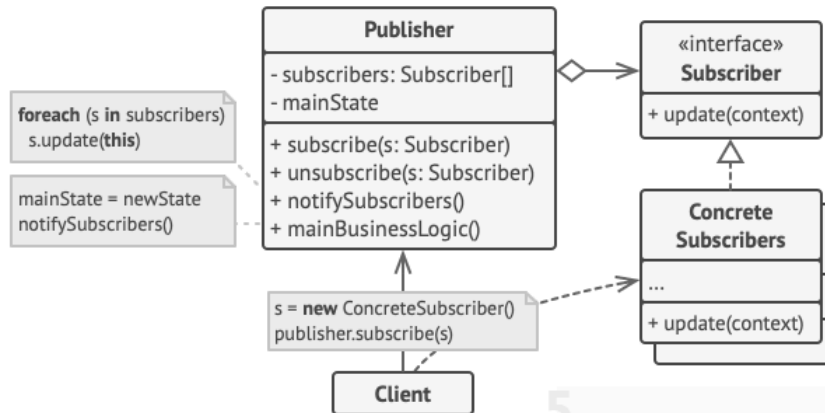
- Когда об изменениях одного объекта нужно уведомлять другие объекты
- Когда количество подписчиков неизвестно и может меняться во время работы программы
- В UI-библиотеках для обработки кликов и событий

5. Структура

1 Издатель владеет внутренним состоянием, изменение которого интересно отслеживать подписчикам. Издатель содержит механизм подписки: список подписчиков и методы подписки/отписки.

2 Когда внутреннее состояние издателя меняется, он оповещает своих подписчиков. Для этого издатель проходит по списку подписчиков и вызывает их метод оповещения, заданный в общем интерфейсе подписчиков.

3 Подписчик определяет интерфейс, которым пользуется издатель для отправки оповещения. В большинстве случаев для этого достаточно единственного метода.



4 Конкретные подписчики выполняют что-то в ответ на оповещение, пришедшее от издателя. Эти классы должны следовать общему интерфейсу подписчиков, чтобы издатель не зависел от конкретных классов подписчиков.

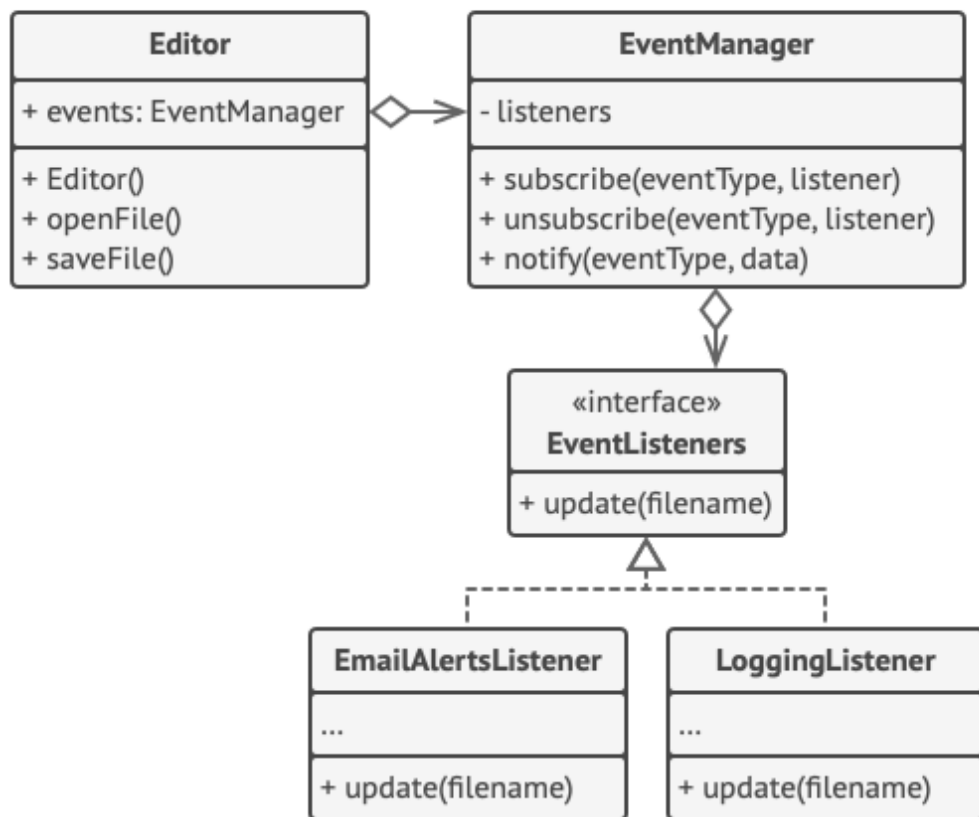
6 Клиент создаёт объекты издателей и подписчиков, а затем регистрирует подписчиков на обновления в издателях.

5 По приходу оповещения подписчику нужно получить обновлённое состояние издателя. Издатель может передать это состояние через параметры метода оповещения. Более гибкий вариант — передавать через параметры весь объект издателя, чтобы подписчик мог сам получить требуемые данные. Как вариант, подписчик может постоянно хранить ссылку на объект издателя, переданный ему в конструкторе.

Участники:

Участник	Описание
Publisher (Издатель)	Хранит список подписчиков, методы subscribe/unsubscribe/notify
Subscriber (Подписчик)	Интерфейс с методом update()
ConcreteSubscribers	Конкретные подписчики, реализуют update()
Client	Создаёт объекты и подписывает их

6. Реализация на Python



Пример оповещения объектов о событиях в других объектах.

Код под структурой с сайта (издатель, подписчики, клиент):

```

from abc import ABC, abstractmethod
from typing import List, Dict

# интерфейс подписчика
class EventListener(ABC):
    @abstractmethod
    def update(self, filename: str) -> None:
        pass

# менеджер событий
class EventManager:
    def __init__(self):
        self._listeners: Dict[str, List[EventListener]] = {}

    def subscribe(self, event_type: str, listener: EventListener) -> None:
        if event_type not in self._listeners:
            self._listeners[event_type] = []
        self._listeners[event_type].append(listener)
  
```

```

        print(f" → Подписан {listener.__class__.__name__} на {event_type}")

    def unsubscribe(self, event_type: str, listener: EventListener) -> None:
        if event_type in self._listeners:
            self._listeners[event_type].remove(listener)
            print(f" → Отписан {listener.__class__.__name__} от
{event_type}")

    def notify(self, event_type: str, data: str) -> None:
        if event_type in self._listeners:
            for listener in self._listeners[event_type]:
                listener.update(data)

# издатель ( ну редактор)
class Editor:
    def __init__(self):
        self.events = EventManager()

    def open_file(self, path: str) -> None:
        print(f"\n[Editor] Открыт файл: {path}")
        self.events.notify("open", path)

    def save_file(self, path: str) -> None:
        print(f"\n[Editor] Сохранён файл: {path}")
        self.events.notify("save", path)

# подписчики
class EmailAlertsListener(EventListener):
    def __init__(self, email: str):
        self._email = email

    def update(self, filename: str) -> None:
        print(f" Email отправлен на {self._email}: файл {filename} изменён")

class LoggingListener(EventListener):
    def __init__(self, log_file: str):
        self._log_file = log_file

    def update(self, filename: str) -> None:
        from datetime import datetime
        print(f" Лог записан в {self._log_file}: {filename} в
{datetime.now()}")

class SMSSender(EventListener):
    def __init__(self, phone: str):
        self._phone = phone

```

```

def update(self, filename: str) -> None:
    print(f"    SMS отправлено на {self._phone}: файл {filename} сохранён")

# клиент
if __name__ == "__main__":
    print("=" * 50)
    print("Демонстрация паттерна Наблюдатель")
    print("=" * 50)

    editor = Editor()

    email = EmailAlertsListener("admin@example.com")
    logger = LoggingListener("app.log")
    sms = SMSSender("+7-999-123-4567")

    print("\n--- Подписка ---")
    editor.events.subscribe("open", email)
    editor.events.subscribe("save", email)
    editor.events.subscribe("save", logger)
    editor.events.subscribe("save", sms)

    editor.open_file("test.txt")
    editor.save_file("test.txt")

    print("\n--- Отписка от save ---")
    editor.events.unsubscribe("save", email)

    editor.save_file("test2.txt")

    print("\n" + "=" * 50)

```

Вывод программы:

```

=====
Демонстрация паттерна Наблюдатель
=====

--- Подписка ---
→ Подписан EmailAlertsListener на open
→ Подписан EmailAlertsListener на save
→ Подписан LoggingListener на save
→ Подписан SMSSender на save

[Editor] Открыт файл: test.txt
📧 Email отправлен на admin@example.com: файл test.txt изменён

```

[Editor] Сохранён файл: test.txt

 Email отправлен на admin@example.com: файл test.txt изменён

 Лог записан в app.log: test.txt в 2026-05-14 15:30:00

 SMS отправлено на +7-999-123-4567: файл test.txt сохранён

--- Отписка от save ---

→ Отписан EmailAlertsListener от save

[Editor] Сохранён файл: test2.txt

 Лог записан в app.log: test2.txt в 2026-05-12 12:30:05

 SMS отправлено на +7-999-123-4567: файл test2.txt сохранён

7. Плюсы и минусы

Плюсы:

- Издатель не зависит от конкретных классов подписчиков
- Можно подписывать/отписывать на лету
- Легко добавлять новых подписчиков

Минусы:

- Подписчики оповещаются в случайном порядке
- Много подписчиков следовательно может тормозить

8. Известные применения

Этот паттерн вообще часто встречается в программировании, я аж сам удивился когда понял.

- **GUI (Swing, JavaFX, Tkinter и т.д.)** - там типа когда на кнопку нажимаешь, она оповещает всех кто её слушает. Я на питоне с Tkinter делал небольшое приложение, там кнопки работают именно так, но я тогда не знал что это паттерн Наблюдатель.
- **Django Signals** - штука которая позволяет одним частям приложения узнавать о том что происходит в других. Например когда пользователь создаётся, можно отправить письмо. Я на Django писал небольшой сайт, использовал сигналы, но тоже не в курсе был что это паттерн.
- **Системы мониторинга** - типа когда сервер падает, он оповещает всех кто подписан, админам приходят уведомления на почту или в телеграм. Ну тут логично, наблюдатель же.
- **ReactiveX (RxPy)** - это вообще про потоки событий, там без наблюдателя никуда. Сам не использовал но на лекциях рассказывали.

В общем паттерн прям везде используется, даже не замечаешь как.

9. Родственные паттерны

- **Посредник** - они похожи но есть разница. Посредник типа собирает всю связь в одном месте, у него есть центральный объект который всё контролирует. А у Наблюдателя издатели и подписчики общаются напрямую, без посредника. Хотя в интернете пишут что их иногда путают. В целом Посредник сложнее как по мне.
- **Издатель-Подписчик** - это почти то же самое но с брокером сообщений между ними. Там типа издатель даже не знает кто подписался, он просто шлёт сообщения брокеру, а тот уже раздаёт. В классическом Наблюдателе издатель сам хранит список подписчиков. Так что это такая улучшенная асинхронная версия. Я решил не усложнять и сделал классический вариант, потому что брокер это уже перебор для лабы.

Также видел что сравнивают с **Цепочкой обязанностей** и **Командой**, но это уже совсем другое, там запрос передаётся по цепочке и обрабатывается кем-то одним, а у нас все подписчики получают уведомление одновременно.

Источники

- Схемы: <https://refactoringu.ru/ru/design-patterns/observer.html>
- Реализация: собственная